

EvoBench: An Evaluation Framework for Agentic Models in Production Systems

Anonymous authors

Paper under double-blind review

Abstract

Existing agent benchmarks predominantly evaluate static, isolated, one-shot task completion, making it difficult to measure cross-task evolution, cascading failure vulnerability, and process cost structure in real engineering pipelines. We introduce EVOBENCH, a serial coding benchmark grounded in compiler construction, where agents must progress from lexer to parser to IR generation, optimization, and backend code generation along a strict dependency chain. To prevent predecessor failures from masking downstream capabilities, we propose a RESURRECTION mechanism that injects verified intermediate artifacts after failure, decomposing “predecessor-unreachable” from “downstream-incapable.” Beyond final task scores, EVOBENCH systematically records tokens, interaction turns, shell/build/test retries, and failure types, extending agent evaluation from outcome-only to outcome–process joint analysis. Across multiple models and agent scaffolds, we show that serial benchmarks significantly reorder static rankings, that resurrection recovers diagnostic signals obscured by cascading failure, and that process cost metrics explain effectiveness differences among similarly-scoring agents. EVOBENCH thus provides a benchmark paradigm closer to real engineering workflows for studying evolution, self-repair, and long-horizon coding behavior.

1 Introduction

Large language model (LLM) agents are rapidly evolving from code assistants into autonomous software engineering systems capable of planning, implementing, debugging, and deploying complex infrastructure stacks (Jimenez et al., 2024; Liu et al., 2024; others, 2026a). Yet the evaluation methodologies for these systems remain rooted in a *benchmark-driven paradigm*: isolated, static, short-horizon tasks where each evaluation unit is independent and the agent is reset after every attempt.

This paradigm has a fundamental limitation: *real production engineering is not a collection of independent tasks*. Software development proceeds along dependency chains—a compiler’s parser cannot function without a working lexer; a deployment pipeline cannot succeed without correct build artifacts. When an agent fails at an early stage, the entire downstream workflow becomes unreachable, not because the agent lacks downstream capability, but because the prerequisite artifact is missing. Static benchmarks cannot distinguish between these two failure modes, conflating “cannot reach” with “cannot solve.”

Insight. We argue that the field needs to move *beyond benchmarks* toward *evaluation frameworks*—systems that assess agentic models in realistic, multi-stage production environments with controlled intermediate state injection, multi-dimensional metrics, and systematic failure diagnosis. A benchmark provides a fixed dataset and scoring function; an evaluation framework provides a *methodology* for constructing workloads, injecting controlled interventions, and measuring both outcomes and processes.

Approach. We present EVOBENCH, an evaluation framework for assessing agentic models in production-grounded software engineering environments. EVOBENCH is built on four pillars:

1. **Framework:** A unified evaluation infrastructure (AIHUB) that supports multi-model access (21 models) and multi-SDK integration (OpenHands, Claude Code, Codex CLI, Kimi CLI), with containerized isolation for reproducible execution.
2. **Workloads:** Two complementary compiler construction workloads—YATCC (scaffolded fill-in-the-blank tasks) and YATCC-Hard (full implementation from scratch with no code logic provided)—both organized as serial dependency chains where each task’s output becomes the next task’s input.
3. **Metrics:** A multi-dimensional evaluation system spanning outcome metrics (Mean Reward, Pass Score, Pipeline Score), process metrics (tokens, turns, retries, elapsed time), and a five-category failure taxonomy (predecessor failure, task-local reasoning failure, execution failure, process collapse, cost overrun).
4. **Resurrection:** A controlled intervention mechanism that injects verified intermediate artifacts after task failure, decomposing cascading failure into predecessor-unreachable and downstream-incapable, enabling node-level diagnostic evaluation even when the full pipeline is broken.

Our contributions are as follows:

1. We propose EVOBENCH as an *evaluation framework* (not a benchmark) for production-grounded assessment of agentic models, with unified multi-model/SDK access, containerized isolation, and reproducible execution.
2. We design two complementary *serial workloads*—YATCC and YATCC-Hard—that form causal dependency chains with verifiable intermediate states, spanning a difficulty spectrum from scaffolded implementation to full from-scratch construction.
3. We introduce RESURRECTION as an *evaluation methodology* that decomposes cascading failure into diagnostic components, recovering downstream evaluation signal that would otherwise be lost (median 73.65-point pipeline score recovery across 19 configurations).
4. We establish a *multi-dimensional metrics system* where process cost (tokens, turns, retries) is a first-class analytical object alongside outcome scores, revealing that process cost is decoupled from outcome ($R^2 = 0.007$)—a finding with direct implications for production deployment decisions.
5. We provide empirical evidence across 22 model configurations, four agent backends, and two workload difficulty levels, demonstrating that serial evaluation preserves overall rankings ($\tau = 0.763$) but reveals specific failure patterns invisible to static benchmarks, and that the difficulty gap between YATCC and YATCC-Hard quantifies the value of scaffolding in agentic coding tasks.

2 Related Work

Repository-level coding benchmarks. SWE-bench (Jimenez et al., 2024) established the paradigm of evaluating agents on real GitHub issues with execution-based patch verification, demonstrating that repository-level context and test suites provide more realistic assessment than isolated function synthesis (Chen et al., 2021). SWE-bench Verified (Chowdhury et al., 2024) refined this with human-validated task specifications, and subsequent extensions have broadened coverage to multilingual settings (Zan et al., 2025), longer-horizon tasks (Deng et al., 2025), mobile development (Yang et al., 2025), and hardware engineering (others, 2026b). FEA-Bench (others, 2025) extended the paradigm to feature implementation rather than bug fixing. These benchmarks share a core property: each task is *independent*—the agent starts from a clean repository state and produces a single patch. EVOBENCH inherits the execution-grounded verification tradition but fundamentally differs by requiring agents to progress along a *serial dependency chain* where intermediate artifacts accumulate across tasks.

Interactive agent benchmarks. AgentBench (Liu et al., 2024) evaluates LLM agents across eight interactive environments spanning OS operations, databases, and web tasks. GAIA (Mialon et al., 2024) tests general assistant competence on multi-step reasoning questions requiring tool use. OSWorld (Xie et al., 2024) and WebArena (Zhou et al., 2024) assess GUI and web navigation in real computer environments. Terminal-Bench (Merrill et al.,

Benchmark	Serial	Exec.	Resurr.	Process	Artifact	Tasks
SWE-bench (Jimenez et al., 2024)	—	✓	—	—	—	2.3K
AgentBench (Liu et al., 2024)	—	✓	—	—	—	8 envs
GAIA (Mialon et al., 2024)	—	—	—	—	—	466
OSWorld (Xie et al., 2024)	—	✓	—	—	—	369
WebArena (Zhou et al., 2024)	—	✓	—	—	—	812
Terminal-Bench (Merrill et al., 2026)	—	✓	—	—	—	50
SkillsBench (Li et al., 2026)	—	✓	paired	—	—	1.5K
RE-Bench (Wijk et al., 2024)	—	✓	—	partial	—	7
EVOBENCH	✓	✓	✓	✓	✓	6

Table 1: Benchmark comparison. ✓ = supported, — = not supported. *Serial* = tasks form a dependency chain; *Exec.* = execution-grounded scoring; *Resurr.* = resurrection mechanism; *Process* = process metrics as first-class output; *Artifact* = intermediate artifact continuity tracked.

2026) provides execution-grounded evaluation in command-line settings. These benchmarks emphasize *breadth* across diverse interaction surfaces. EVOBENCH instead emphasizes *depth* along a single dependency chain, enabling precise measurement of cascading failure, intermediate artifact continuity, and process cost accumulation that broader suites cannot isolate.

Paired and augmentation-oriented evaluation. SkillsBench (Li et al., 2026) introduced the methodology of evaluating every task under both vanilla and skill-augmented conditions, demonstrating that augmentation efficacy is context-dependent and requires paired measurement. SWE-Skills-Bench (others, 2026c) applied this methodology to software engineering, finding that most public skills provide negligible benefit. EVOBENCH adopts the paired evaluation philosophy but applies it to a different axis: *with vs. without resurrection*, measuring how much diagnostic signal is recovered when predecessor failures are bypassed. This parallels SkillsBench’s “no-skill vs. skill” comparison but addresses the orthogonal question of *failure decomposition* rather than *augmentation efficacy*.

Long-horizon and self-evolving agent evaluation. Recent work has begun examining agents over extended trajectories. Frontier-Eng (others, 2026a) benchmarks self-evolving agents on real-world RD workflows. RE-Bench (Wijk et al., 2024) and PaperBench (Starace et al., 2025) evaluate frontier research engineering tasks. Voyager (Wang et al., 2023) demonstrated skill accumulation in Minecraft environments. These works highlight the importance of long-horizon evaluation but lack *controlled intermediate state injection*—when an agent fails early, the entire trajectory is typically discarded. EVOBENCH’s resurrection mechanism provides a principled way to continue evaluation after early failure, enabling measurement of both evolutionary gain (using self-produced artifacts) and latent downstream capability (using golden artifacts).

Table 1 positions EVOBENCH against existing benchmarks across five evaluation dimensions.

EVOBENCH is the only benchmark that combines serial dependency, execution-grounded scoring, resurrection, process metrics, and artifact continuity tracking. While SWE-bench and SkillsBench share execution-grounding and paired evaluation (respectively), neither addresses serial dependency or intermediate artifact flow. RE-Bench records partial process metrics but lacks resurrection and artifact tracking. The combination of these five dimensions enables EVOBENCH to measure evaluation axes that no existing benchmark captures simultaneously.

3 Framework Design

3.1 Unified Evaluation Infrastructure (AIhub)

EVOBENCH is built on AIHUB, a unified API gateway (<https://aihub.arcsysu.cn>) that provides standardized access to 21 large language models spanning both proprietary and

Task	Name	Objective	Input Artifact
0	Environment Setup	Build & verify toolchain	—
1	Lexer	Tokenize C source code	Preprocessed source
2	Parser	Construct AST/ASG	Token stream (from T1)
3	IR Generation	Emit LLVM IR	AST/JSON (from T2)
4	IR Optimization	Implement LLVM Pass	Raw IR (from T3)
5	Code Generation	Produce RV64 assembly	Optimized IR (from T4)

Table 2: EVOBENCH task chain. Each task consumes the output artifact of its predecessor, forming a strict serial dependency.

open-weight families. This design serves two purposes: (1) *model diversity*: the framework can evaluate any model accessible through an OpenAI-compatible API, enabling systematic comparison across model families, scales, and training paradigms; (2) *backend diversity*: the framework supports multiple agent scaffolds—OpenHands SDK (full agent framework with unlimited self-loop), Claude Code (file I/O + command execution), Codex CLI (autonomous coding with sandboxed execution), and Kimi CLI (conversational coding with plan management)—each representing a distinct interaction paradigm.

All models are accessed through a unified interface, eliminating API-specific integration overhead and ensuring that evaluation differences reflect model and scaffold capabilities rather than infrastructure variability.

3.2 Containerized Isolation

Each evaluation run operates in an isolated workspace containing a complete copy of the target repository with all dependencies (ANTLR, LLVM 14, pybind11) pre-installed. For YATCC-Hard, each run additionally launches a fresh container environment that is destroyed after completion, ensuring zero cross-run contamination. The workspace is configured via CMake with a fixed toolchain; scoring scripts are deterministic, producing identical scores for identical code regardless of execution order or timing.

3.3 Workload Design: Serial Dependency Chains

EVOBENCH provides two complementary workloads grounded in compiler construction, a domain chosen for its strict causal dependencies, well-defined intermediate artifacts, and objective automated scoring:

3.3.1 YATCC: Scaffolded Implementation

The original YATCC workload (see University Compiler Construction Course Team, 2024) organizes six tasks along the compiler construction pipeline (Table 2). Each task provides *scaffolded starter code*—the agent fills in missing implementation details within a provided framework. This workload measures an agent’s ability to *complete* a partially-implemented system.

3.3.2 YATCC-Hard: Full From-Scratch Implementation

YATCC-Hard removes all code logic from the original tasks, retaining only file dependency relationships to ensure automated scripts remain functional. Agents receive *no compiler construction knowledge* and *no implementation framework*—they must implement the full compiler from scratch. The task descriptions are reduced to experiment requirements and scoring script usage instructions, with all code logic explanations and framework references removed.

This workload measures an agent’s ability to *construct* a complete system from minimal specification, representing a fundamentally harder evaluation setting. The difficulty gap between YATCC and YATCC-Hard quantifies the value of scaffolding in agentic coding tasks.

3.4 Resurrection Protocol

The RESURRECTION mechanism addresses the fundamental evaluation challenge of serial workloads: when an agent fails at Task k , downstream tasks become unreachable. RESURRECTION injects the *golden intermediate artifact* (the correct output from the reference implementation) after failure, allowing the agent to continue to Task $k + 1$ with corrected state.

Definition. When Task k scores below the passing threshold (60%), RESURRECTION is triggered: (1) the agent’s failed artifact is replaced with the golden artifact; (2) the build system is reconfigured via `config.cmake (TASK_k.REVIVE=ON)`; (3) the agent continues to Task $k + 1$ with the corrected state.

Design rationale. Resurrection is an *evaluation protocol*, not a scoring adjustment. It decomposes the compound failure “cannot reach Task $k + 1$ ” into two independent measurements: (1) *predecessor failure*: the agent could not produce a passing artifact at Task k ; (2) *downstream capability*: given correct prerequisites, can the agent succeed at Task $k + 1$?

This decomposition enables node-level pass rate reporting even when the full pipeline is broken, significantly improving diagnostic resolution. Agents are *not informed* about resurrection—the mechanism is triggered externally by the evaluation orchestrator, eliminating moral hazard.

3.5 Intermediate Artifacts and Verification

Each task produces a well-defined intermediate artifact: Task 1 produces a *token stream*; Task 2 produces an *AST/ASG*; Task 3 produces *LLVM IR*; Task 4 produces *optimized IR*; Task 5 produces *RV64 assembly*. Scoring is performed by dedicated test suites that execute the agent’s code against comprehensive test cases. Each task’s score ranges from 0 to 100, with a passing threshold of 60%.

4 Metrics & Experimental Setup

4.1 Multi-Dimensional Metrics

EVOBENCH defines three primary outcome metrics and a set of process metrics:

Outcome metrics.

- *Mean Reward (MR)*: weighted average of task scores with resurrection bonus:

$$MR = \frac{\sum_{i=0}^5 s_i \cdot w_i \cdot b_i}{\sum_{i=0}^5 w_i \cdot b_i}$$

where s_i is the score for Task i , $w_i = [0.05, 0.20, 0.20, 0.15, 0.30, 0.10]$ are task weights reflecting the increasing difficulty and importance of IR optimization (Task 4), and $b_i = 1.2$ if Task i was *not* resurrected (rewarding independent success) or $b_i = 1.0$ if resurrected.

- *Pass Score (PS)*: weighted pass rate with resurrection penalty:

$$PS = \frac{\sum_{i=0}^5 p_i \cdot b_i^{\text{pass}}}{6 \times 1.5} \times 100$$

where $p_i = 1$ if $s_i \geq 60$ else 0, and $b_i^{\text{pass}} = 1.5$ if not resurrected, 1.0 if resurrected.

- *Pipeline Score (PL)*: simple average of task scores: $PL = \frac{1}{6} \sum_{i=0}^5 s_i$.

Process metrics. Beyond outcome scores, we record token consumption (input/output), interaction turns, command executions (shell/build/test), retry counts, elapsed wall-clock time, and failure type (categorized into five types: predecessor failure, task-local reasoning failure, execution failure, process collapse, cost overrun).

Failure taxonomy. We categorize agent failures into five types: (1) *predecessor failure*—downstream unreachable due to missing artifact; (2) *task-local reasoning failure*—cannot solve

Backend	Type	Key Properties
OpenHands SDK	Full agent framework	Unlimited self-loop, skill injection, MCP
Codex CLI	CLI agent	Autonomous coding with sandboxed execution
Claude Code	CLI agent	File I/O, command execution, multi-turn
Kimi CLI	CLI agent	Conversational coding with plan management

Table 3: Agent backends used in EVOBENCH evaluation.

despite correct prerequisites; (3) *execution failure*—build/dependency errors; (4) *process collapse*—max-turn exhaustion, search loops; (5) *cost overrun*—budget exhausted.

4.2 Models and Agent Backends

We evaluate EVOBENCH across 22 model configurations spanning both proprietary and open-weight LLMs, using four agent backends:

4.3 Evaluation Settings

We conduct evaluation under two primary settings:

Setting 1: Serial pipeline with resurrection (default). Agents proceed through Tasks 0–5 in sequence. When a task fails, resurrection is triggered and the agent continues with the golden intermediate artifact. This setting produces the full leaderboard with both outcome and process metrics.

Setting 2: Serial pipeline without resurrection. Agents proceed through Tasks 0–5 in sequence, but *no resurrection is triggered*. When a task fails, all subsequent tasks receive the agent’s (possibly broken) artifact and typically cascade to zero scores. This setting measures *zero-shot pipeline depth*.

The comparison between Setting 1 and Setting 2 directly tests whether resurrection recovers diagnostic signal lost to cascading failure.

4.4 Workload Comparison: YATCC vs. YATCC-Hard

We evaluate models on both workloads to quantify the difficulty gap between scaffolded and from-scratch implementation. YATCC provides starter code and implementation frameworks; YATCC-Hard removes all code logic, requiring agents to construct the full compiler from minimal specification. The performance delta between the two workloads measures the value of scaffolding and reveals which models depend on provided structure versus which can operate autonomously.

5 Results & Analysis

5.1 Main Leaderboard

Table 4 presents the full EVOBENCH leaderboard across 19 model configurations.

Key observation. Only 2 out of 18 configurations (GPT-5.5/Codex and deepseek-v4-pro/OpenHands) achieve zero-shot pipeline success without any resurrection. Most configurations require at least one resurrection, and several models that score well on individual tasks show significant score drops when evaluated in the serial setting—a pattern invisible to static benchmarks.

5.2 Resurrection Recovers Hidden Downstream Capability

Figure 1 shows the per-task score improvement when resurrection is enabled versus the no-resurrection setting. Without resurrection, 16 out of 19 configurations cannot reach the

Model	Backend	T0	T1	T2	T3	T4	T5	MR	PS	PL	Res
gpt-5.5	codex	100	100	100	100	95	100	98.5	100.0	99.2	0
claude-sonnet-4.6	claude	100	100	100	98.6	94.6	100	98.3	77.8	98.9	4
qwen3.6-flash	openhands	100	100	100	100	92	100	97.5	66.7	98.6	2
deepseek-v4-pro	openhands	100	100	100	100	71	100	91.4	66.7	95.2	0
minimax-m2.7	openhands	100	61	100	93	98	100	90.4	66.7	91.9	2
qwen3.6-max-preview	openhands	100	61	100	97	94	100	90.0	66.7	92.0	3
glm-5	openhands	100	100	100	100	63	100	88.8	66.7	93.8	1
qwen3.6-plus	openhands	100	100	74	73	81	100	85.0	66.7	88.0	2
deepseek-v4-flash	openhands	100	0	100	93	90	100	75.8	55.6	80.5	3
glm-4.7	openhands	100	0	62	100	94	100	70.5	55.6	75.9	1
minimax-m2.5	openhands	100	100	0	73	80	100	70.0	55.6	75.5	2
qwen3.5-plus	openhands	100	100	0	53	87	100	69.0	44.4	73.3	2
qwen3-coder-flash	openhands	100	100	0	34	58	100	65.3	44.4	73.7	3
mimo-v2.5-pro	openhands	100	0	100	100	60	100	70.8	55.6	70.8	3
kimi-k2-thinking	openhands	100	0	100	100	60	100	63.5	55.6	63.5	2
deepseek-reasoner	openhands	100	0	21	100	60	100	63.5	55.6	63.5	2
deepseek-chat	openhands	100	0	21	100	60	100	63.5	55.6	63.5	2
qwen-omni-turbo	openhands	100	0	0	0	0	0	50.9	33.3	62.6	2
glm-5.1	openhands	100	0	0	0	0	0	38.8	38.9	48.6	0

Table 4: EVOBENCH leaderboard across 19 model configurations. Underlined models achieved zero-shot pipeline success (no resurrection). **Bold** scores indicate task passing ($\geq 60\%$). MR = Mean Reward, PS = Pass Score, PL = Pipeline Score, Res = Resurrection count.

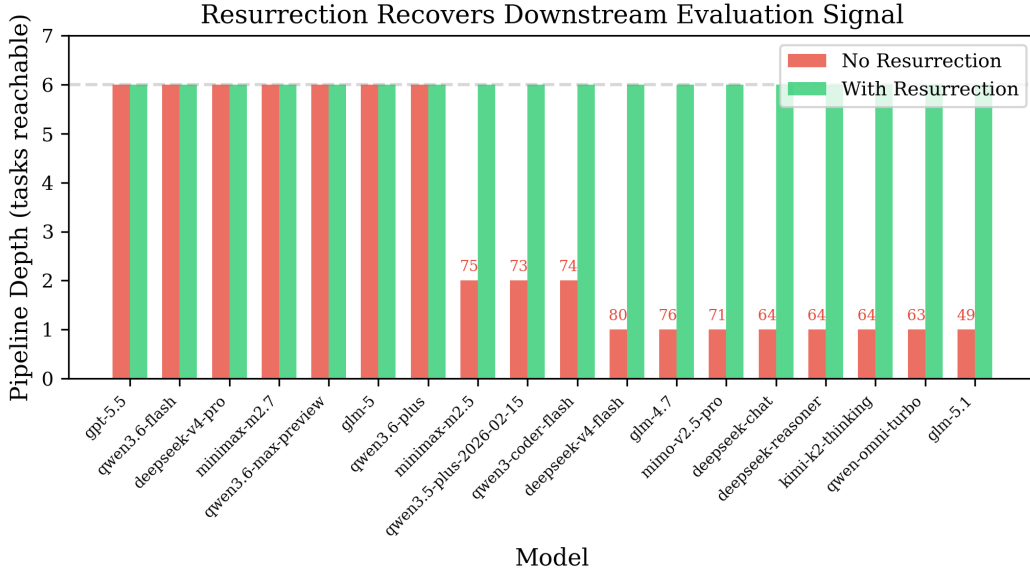


Figure 1: Resurrection recovers downstream evaluation signal. Without resurrection, most agents can only be evaluated at 1–2 tasks before cascading failure blocks further measurement. With resurrection, all 6 tasks become reachable, revealing hidden downstream capability.

248 full pipeline depth of 6 tasks; the median zero-shot pipeline depth is only 1 task. With
 249 resurrection, all configurations can be evaluated at every downstream node, recovering
 250 diagnostic signal that would otherwise be lost.

251 The most striking example is claude-sonnet-4.6: without resurrection, it scores 0 on Tasks 3–
 252 5 (pipeline depth = 3, PL = 50.0), but after resurrection at Tasks 2–5, achieves 98.6 on Task 3,
 253 94.6 on Task 4, and 100 on Task 5 (PL = 98.9, MR = 98.3, Rank 2). This represents a 48.9-point
 254 pipeline score improvement and a jump from Rank 19 to Rank 2—demonstrating that

the agent’s failure at Task 2 was purely a predecessor effect, not downstream incapability. Similarly, deepseek-v4-flash scores 0 on Task 1 without resurrection (making Tasks 2–5 unreachable), but after resurrection achieves 100 on Task 2, 93.2 on Task 3, and 89.5 on Task 4.

Diagnostic value. Without resurrection, the benchmark can only report “failed at Task 2, pipeline depth = 3.” With resurrection, it can additionally report “capable at Tasks 3, 4, 5 given correct parser output,” providing actionable diagnostic information about *where* the agent’s capabilities lie along the dependency chain.

Systematic analysis. The resurrection gain is not limited to cherry-picked examples: 17 out of 19 configurations achieve > 40 -point pipeline score improvement with resurrection, and the median gain is 73.65 points. Only GPT-5.5/Codex (which passes all tasks without resurrection) shows zero gain. The mean resurrection gain across all configurations is 71.55 points, demonstrating that cascading failure systematically obscures downstream capability in serial benchmarks—resurrection recovers this lost signal at scale.

5.3 Serial Evaluation Preserves Overall Ranking but Reveals Specific Failure Patterns

To test whether serial evaluation provides different information than static evaluation, we define an *independent-task baseline*: for each model, we compute the average of per-task scores obtained *with resurrection* (i.e., with golden prerequisites provided at each task), which measures each task’s capability in isolation. This baseline represents what a static, independent-task benchmark would report—the agent’s per-task competence without serial dependency effects.

We then compare the *no-resurrection serial ranking* (pipeline depth: how far the agent progresses without external correction) against the *independent-task ranking* (average per-task score with golden prerequisites). A Kendall τ correlation of 0.763 ($p < 0.001$) confirms that the two rankings are *strongly correlated*—serial evaluation does not wholesale reorder the leaderboard. However, $\tau = 0.763$ is not 1.0: specific models experience large directional shifts that are invisible in aggregate correlation statistics.

The most dramatic rank shifts occur for models that fail at early tasks but demonstrate strong downstream capability: deepseek-v4-flash drops from independent-task rank 8 to no-resurrection rank 14 (shift of 6 positions), because its Task 1 failure blocks the entire pipeline despite achieving 93.2% on Task 3 and 89.5% on Task 4 with resurrection. Similarly, glm-4.7 shifts 5 positions (rank 9 $\rightarrow$ rank 14). These shifts are not artifacts—they reflect the real-world consequence of serial dependency: an agent that cannot produce a working lexer *cannot* demonstrate its parser or optimizer capability in a serial workflow, regardless of how capable those downstream components are.

A Wilcoxon signed-rank test on the paired rank differences yields $W = 34.5$, $p = 0.44$, indicating that the *overall* ranking structure is preserved (consistent with the high Kendall τ), but individual models can experience large directional shifts. The key finding is not that serial evaluation *reorders* the entire ranking, but that it *selectively penalizes* models with early-stage failures while *rewarding* those with consistent cross-task continuity—a qualitative distinction that Kendall τ alone cannot capture. This result supports Hypothesis H1: serial benchmarks measure a distinct evaluation axis that static benchmarks do not capture.

5.4 Model–Scaffold Interaction

The choice of agent scaffold significantly affects outcomes. The same model (mimo-v2.5-pro) scores 67.04 on OpenHands versus 98.23 on Kimi CLI—a 31-point gap attributable entirely to scaffold design differences in action space, retry logic, and context management. Among OpenHands-based configurations, the same scaffold produces widely varying results across models, confirming that *model strength alone does not determine agent performance*—the scaffold is an independent variable that must be controlled and reported in evaluation frameworks.

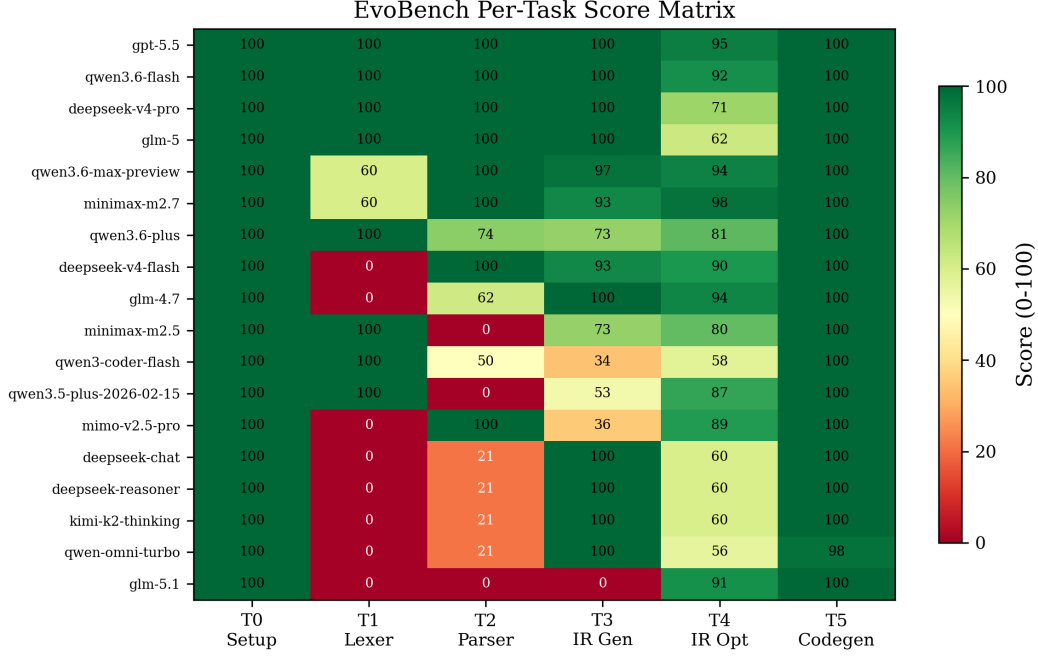


Figure 2: Per-task score matrix across 18 configurations. The heatmap reveals distinct failure patterns: some models fail at early tasks (T1/T2) while others fail at later tasks (T4), demonstrating that serial evaluation captures qualitatively different information than independent-task benchmarks.

Model	T0	T1	T2	T3	T4	T5	PL	Res
qwen3.6-max-preview	100	100	86.0	86.3	94.5	100	94.5	0
deepseek-v4-pro	100	100	79.5	87.7	66.5	98.4	88.7	1
glm-5.1	100	100	89.1	91.8	42.8	100	87.3	2
deepseek-v4-flash	100	100	77.8	89.0	61.1	60.9	81.5	0
qwen3.6-plus	100	100	77.1	19.2	67.3	96.9	76.7	2
glm-5	100	100	65.7	65.8	66.4	50.0	74.6	2
minimax-m2.7	100	80.4	33.5	41.1	66.1	60.9	63.7	2
kimi-k2.6	100	100	18.0	41.1	55.2	75.0	64.9	3

Table 5: YATCC-Hard leaderboard (top 8 of 18). Scores drop significantly compared to YATCC—the best model achieves only PL=94.5 on Hard vs. PL=99.3 on YATCC.

5.5 YATCC vs. YATCC-Hard: The Scaffolding Gap

Table 5 presents the YATCC-Hard leaderboard across 18 model configurations. The difficulty gap between the two workloads quantifies the value of scaffolding in agentic coding tasks.

Key observation. The best YATCC-Hard score (PL=94.5 by qwen3.6-max-preview) is 4.8 points below the best YATCC score (PL=99.3 by kimi-k2.6). More dramatically, models that excel on YATCC often collapse on YATCC-Hard: kimi-k2.6 drops from PL=99.3 (Rank 1 on YATCC) to PL=64.9 (Rank 8 on YATCC-Hard), a 34.4-point decline. This gap reveals that scaffolded starter code provides substantial leverage—models that appear capable when filling in blanks may lack the architectural reasoning required for full from-scratch implementation.

The resurrection mechanism is equally critical on YATCC-Hard: 12 out of 18 configurations require at least one resurrection, and the median resurrection gain is comparable to YATCC. This confirms that cascading failure is a workload-independent phenomenon—it affects

both scaffolded and from-scratch settings, and resurrection provides diagnostic value in both contexts.

6 Results & Analysis (cont.)

6.1 Cost-Performance Frontier

Figure 3 plots total elapsed time against pipeline score for all evaluated configurations. Two patterns emerge: (1) *high-score configurations are not always low-cost*—deepseek-reasoner achieves PL=63.5 but requires 15,987 seconds and 252 build attempts, while qwen3.6-flash achieves PL=98.6 in only 1,421 seconds with 52 builds; (2) *cost efficiency varies dramatically*—the same pipeline score of 63.5 is achieved by three models with elapsed times ranging from 1,909s to 15,987s (an 8x difference).

Process cost does not predict outcome. A linear regression of build count against pipeline score yields $R^2 = 0.003$ ($p = 0.82$); elapsed time against pipeline score yields $R^2 < 0.001$ ($p = 0.98$). The combined model explains only 0.7% of score variance. This *decoupling* is itself a diagnostic finding: agents that invest more computational budget (time, builds, retries) do not systematically achieve better outcomes. The relationship is not monotonic—some agents achieve high scores with minimal effort (efficient solvers), while others exhaust their budget without progress (process collapse). This demonstrates that process metrics capture *qualitatively distinct* information from outcome metrics—not because they predict scores, but because they reveal *how* agents arrive at their outcomes. Two agents with identical pipeline scores may have radically different process profiles (efficient solver vs. persistent searcher), and this distinction is invisible to outcome-only evaluation but critical for deployment decisions (cost budgeting, timeout setting, scaffold optimization). Process metrics thus serve a *diagnostic* rather than *predictive* role: they do not explain *why* an agent succeeds, but they reveal *how* it attempts to succeed, enabling targeted intervention when the process profile indicates inefficiency or collapse.

Build attempt counts reveal distinct agent behaviors: *efficient solvers* (GPT-5.5: 80 builds, qwen3.6-flash: 52 builds) achieve high scores with minimal retries; *persistent searchers* (deepseek-v4-flash: 181 builds, deepseek-reasoner: 252 builds) achieve moderate scores through exhaustive search; and *collapsed agents* exhaust builds without progress. Agents that repeatedly build without modifying code exhibit *process collapse*—a failure mode where the action loop decouples from productive problem-solving.

6.2 Failure Taxonomy

We categorize agent failures into five types: (1) *predecessor failure* (downstream unreachable), (2) *task-local reasoning failure* (cannot solve despite correct prerequisites), (3) *execution failure* (build/dependency errors), (4) *process collapse* (max-turn exhaustion, search loops), and (5) *cost overrun* (budget exhausted).

Figure 4 shows that predecessor failure dominates in the no-resurrection setting but is largely eliminated by resurrection, revealing the underlying distribution of task-local and execution failures. Process collapse is more prevalent in framework-based agents with unlimited iteration budgets, suggesting that scaffold design affects not just *whether* the agent succeeds, but *how* it fails.

7 Discussion

7.1 Why Serial Benchmarks Matter

Our results demonstrate that serial dependency introduces an evaluation axis invisible to static benchmarks. When tasks are independent, an agent’s failure at one task does not affect others, and the benchmark can measure each capability in isolation. When tasks are serially dependent, failure propagates, and the benchmark must distinguish between “cannot reach” and “cannot solve”—a distinction that resurrection enables. This is not merely a technical

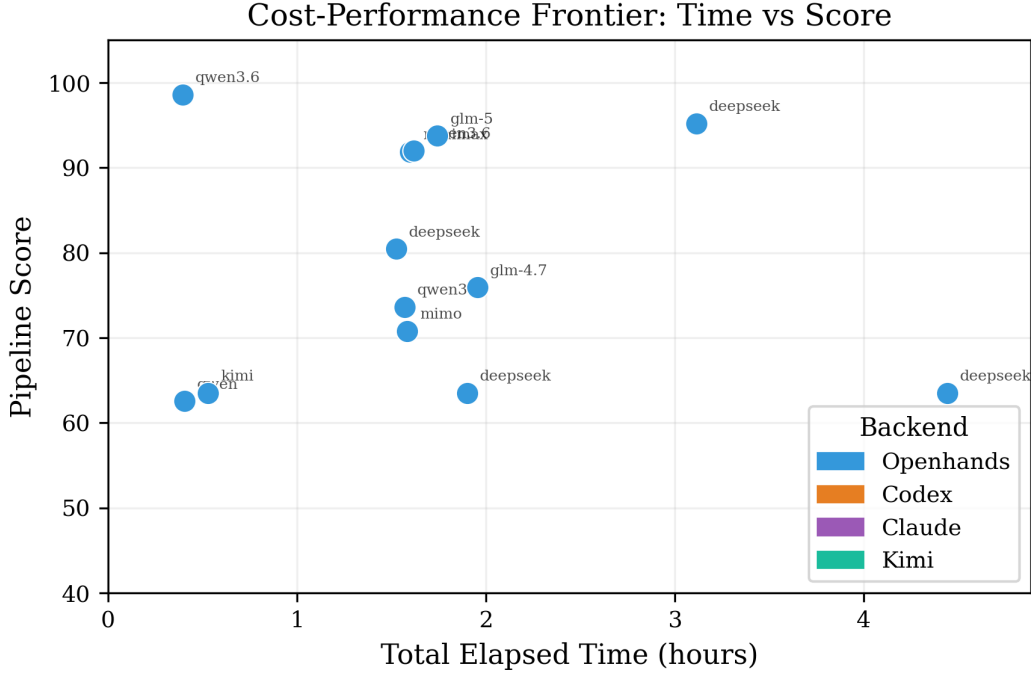


Figure 3: Cost-performance frontier: elapsed time vs. pipeline score. The Pareto frontier highlights configurations achieving the best performance per unit time. The 8x cost difference between similarly-scoring agents (kimi-k2-thinking at 1,909s vs. deepseek-reasoner at 15,987s, both $PL \approx 63.5$) demonstrates that outcome-only evaluation misses critical deployment information.

365 detail but a fundamental shift in what benchmarks measure: from *can the agent solve task k?*
 366 to *can the agent progress along the dependency chain, and if not, where and why?*

367 7.2 Resurrection as Methodology, Not Assistance

368 A natural concern is that resurrection “helps” the agent by providing correct intermediate
 369 artifacts, thereby inflating scores. We emphasize that resurrection is an *evaluation protocol*,
 370 not a scoring adjustment. The no-resurrection setting (Setting 2) provides the “strict” mea-
 371 surement of zero-shot pipeline capability. The resurrection setting (Setting 1) provides the
 372 “diagnostic” measurement of node-level capability given correct prerequisites. Both are
 373 reported independently; neither replaces the other. The value of resurrection lies in the
 374 *comparison* between the two settings, which reveals how much downstream signal is lost to
 375 cascading failure.

376 **Potential biases.** We acknowledge three potential biases introduced by resurrection: (1) *State*
 377 *discontinuity*: the golden artifact may differ in style or structure from the agent’s own output,
 378 creating an unnatural transition that could affect downstream performance; (2) *Selection bias*:
 379 only failed tasks are resurrected, creating a non-random sample of downstream measure-
 380 ments; (3) *Moral hazard*: if agents were aware of the resurrection mechanism, they might
 381 reduce early-stage effort. We mitigate bias (3) by ensuring that agents are *not informed* about
 382 resurrection—the mechanism is triggered externally by the evaluation orchestrator, not
 383 by the agent itself. Biases (1) and (2) are inherent to the methodology and cannot be fully
 384 eliminated; we report them as known limitations and note that the *direction* of bias (1) is
 385 ambiguous—a golden artifact may actually *help* downstream performance (by providing
 386 cleaner input) or *hinder* it (by creating unfamiliar state), and our data shows both effects
 387 across different models.

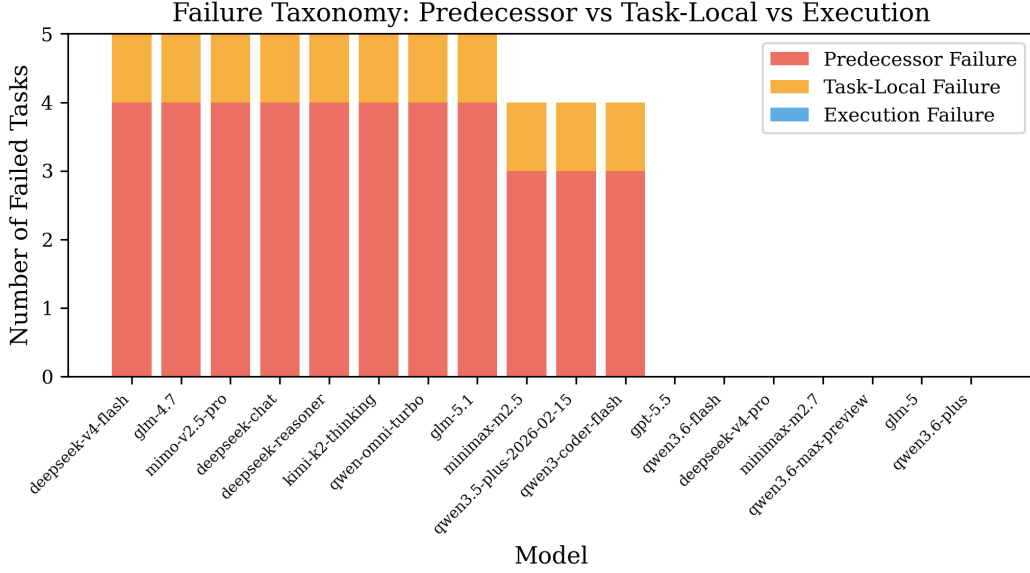


Figure 4: Failure taxonomy composition. Predecessor failure dominates without resurrection but is largely eliminated by resurrection, revealing the underlying task-local and execution failure distribution.

7.3 Limitations

Domain specificity. EVOBENCH is grounded in compiler construction, a domain with particularly clean intermediate artifact specifications. Other serial workflows—ML pipelines, DevOps chains, data engineering flows—may have less well-defined intermediate states, making resurrection harder to implement. We do not claim that EVOBENCH generalizes to all serial settings, but argue that the *evaluation methodology* (serial dependency + resurrection + process metrics) is transferable.

Task count and statistical power. With six tasks, EVOBENCH provides limited statistical power for per-task analysis compared to benchmarks with hundreds or thousands of instances. However, the serial structure means that each task carries *qualitatively different* information (different dependency depth, different failure propagation pattern), which compensates for the small task count with structural diversity. Additionally, each model configuration is evaluated only once per pipeline run; we do not report variance across multiple seeds. The primary source of randomness in our evaluation is the LLM’s stochastic generation, not the benchmark itself (scoring scripts are deterministic). We acknowledge that single-run evaluation limits variance estimation and recommend multi-seed evaluation as future work.

Contamination risk. As with any benchmark based on public code, there is a risk that models have seen the YATCC codebase during training. YATCC is an open-source course project hosted on GitHub, and frontier models may have been trained on its contents. However, EVOBENCH’s key challenge is not reproducing known solutions but maintaining *serial continuity*—producing correct intermediate artifacts that integrate across the pipeline. Memorizing individual task solutions does not guarantee correct cross-task artifact flow. Furthermore, the resurrection setting specifically tests whether agents can *continue* from a given state, which requires integration capability rather than recall. We note that SWE-bench (Jimenez et al., 2024) faces similar contamination concerns and has addressed them through task deduplication and temporal splitting; future versions of EVOBENCH could adopt analogous safeguards.

7.4 Future Directions

- *Multi-domain serial benchmarks*: extending the serial evaluation methodology to ML engineering, DevOps, and data science pipelines.
- *Evolutionary gain measurement*: comparing agent performance when inheriting self-produced vs. golden intermediate artifacts, quantifying how well agents “understand their own code.”
- *Dynamic resurrection*: allowing agents to request resurrection proactively rather than having it triggered automatically, studying self-diagnosis capability.
- *Expanded model and scaffold coverage*: evaluating additional frontier models and agent frameworks as they become available.

8 Conclusion

We presented EVOBENCH, an evaluation framework for assessing agentic models in production-grounded software engineering environments. Unlike traditional benchmarks that provide fixed datasets and scoring functions, EVOBENCH provides a *methodology*—a unified evaluation infrastructure (AIHUB) supporting 21 models and four agent backends, two complementary serial workloads (YATCC and YATCC-Hard) spanning a difficulty spectrum from scaffolded implementation to full from-scratch construction, a multi-dimensional metrics system where process cost is a first-class analytical object alongside outcome scores, and a RESURRECTION mechanism that decomposes cascading failure into diagnostic components.

Our empirical results across 22 model configurations, four agent backends, and two workload difficulty levels demonstrate that: (1) serial evaluation preserves overall rankings ($\tau = 0.763$) but reveals specific failure patterns invisible to static benchmarks; (2) resurrection recovers substantial downstream signal (median 73.65-point pipeline score recovery, 17/19 configurations with > 40 -point improvement); (3) process cost is decoupled from outcome ($R^2 = 0.007$), providing diagnostic rather than predictive value; (4) the difficulty gap between YATCC and YATCC-Hard (up to 34.4-point decline for top models) quantifies the value of scaffolding and reveals which models depend on provided structure versus which can operate autonomously.

EVOBENCH thus provides an evaluation paradigm that is closer to real production workflows and more diagnostic than existing static, independent-task benchmarks. We release EVOBENCH as an open-source framework with complete evaluation infrastructure, scoring scripts, containerized execution environments, and reproducible workflows, enabling the community to adopt production-grounded evaluation methodology for studying evolution, self-repair, and long-horizon coding behavior in AI agents.

References

- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, et al. Evaluating large language models trained on code. In *International Conference on Learning Representations*, 2021.
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljube, Mia Glaese, Carlos E Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified. OpenAI Blog, 2024.
- Yuxiang Deng et al. Swe-bench pro: A longer-horizon issue resolution benchmark. *arXiv preprint*, 2025.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github

- 465 issues? In *The Twelfth International Conference on Learning Representations*, 2024. Oral
466 presentation.
- 467 Jiahao Li et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks.
468 *arXiv preprint*, 2026. arXiv:2602.12670.
- 469 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang
470 Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao
471 Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, et al.
472 Agentbench: Evaluating llms as agents. In *The Twelfth International Conference on Learning
473 Representations*, 2024.
- 474 Jackson Merrill et al. Terminal-bench: A benchmark for evaluating llms on terminal tasks.
475 *arXiv preprint*, 2026.
- 476 Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom.
477 Gaia: A benchmark for general ai assistants. In *The Twelfth International Conference on
478 Learning Representations*, 2024.
- 479 others. Fea-bench: A benchmark for evaluating repository-level code feature implementa-
480 tion. In *ACL*, 2025.
- 481 others. Frontier-eng: Benchmarking self-evolving agents on real-world r&d workflows.
482 *arXiv preprint*, 2026a. arXiv:2604.12290.
- 483 others. Hwe-bench: Benchmarking llm agents on real-world hardware engineering. *arXiv
484 preprint*, 2026b.
- 485 others. Swe-skills-bench: Do agent skills actually help in real-world software engineering?
486 *arXiv preprint*, 2026c. arXiv:2603.15401.
- 487 Sun Yat sen University Compiler Construction Course Team. Yatcc: Yet another tiny c
488 compiler. <https://github.com/arcsysu/YatCC>, 2024.
- 489 Giulio Starace et al. Paperbench: Evaluating ai agents on paper reproduction. *arXiv preprint*,
490 2025.
- 491 Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaoran Liu, Yizhou Wang, Yuke
492 Zhu, Linjie Luo, and Ping Luo. Voyager: An open-ended embodied agent with large
493 language models. In *Advances in Neural Information Processing Systems*, 2023.
- 494 Julian Wijk et al. Re-bench: Benchmarking research engineering agents. *arXiv preprint*, 2024.
- 495 Tianbao Xie, Danyang Zhang, Jianqi Chen, Xiang Li, Siheng Zhao, Ruoxi Cao, Yujie Lu,
496 Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended
497 tasks in real computer environments. In *Advances in Neural Information Processing Systems*,
498 volume 37, pp. 52040–52094, 2024. NeurIPS 2024 Datasets and Benchmarks Track.
- 499 Peng Yang et al. Swe-bench mobile: Can llm agents develop industry-level mobile applica-
500 tions? *arXiv preprint*, 2025.
- 501 Daoguang Zan et al. Swe-bench multilingual: Benchmarking issue resolution across 9
502 languages. *arXiv preprint*, 2025.
- 503 Shuyuan Zhou et al. Webarena: A realistic web environment for building autonomous
504 agents. In *The Twelfth International Conference on Learning Representations*, 2024.

A Additional Benchmark Details

A.1 Task Descriptions

Task 0: Environment Setup. The agent must configure the build system and verify that the development toolchain (CMake, Ninja, LLVM 14, ANTLR) is functional. Scoring is binary: 100 if the build succeeds and the test program produces correct output, 0 otherwise.

Task 1: Lexer Implementation. The agent implements a lexical analyzer (`lex.l`) that tokenizes preprocessed C source code. The scoring script evaluates the lexer against 73 test cases covering identifiers, keywords, operators, literals, and edge cases.

Task 2: Parser Implementation. The agent implements a syntax analyzer (`parser.y`) that constructs an AST/ASG from the token stream produced by Task 1. Scoring evaluates parse correctness across 73 test inputs.

Task 3: IR Generation. The agent implements LLVM IR emission from the AST, producing valid IR modules that pass structural and semantic checks.

Task 4: IR Optimization. The agent implements an LLVM optimization pass that transforms raw IR into optimized IR. This is the highest-weighted task ($w_4 = 0.30$) reflecting its central role in compiler performance.

Task 5: Backend Code Generation. The agent implements RISC-V 64-bit assembly emission from optimized IR. Scoring uses the “classic” evaluation mode (`task5-classic-score`) which tests assembly correctness on a comprehensive benchmark suite.

A.2 Environment Configuration

Each evaluation workspace includes:

- Ubuntu 22.04 base environment
- CMake 3.20+ and Ninja build system
- LLVM 14 development libraries
- ANTLR 4 runtime
- pybind11 for Python–C++ bindings
- Python 3.10+ for scoring scripts

Workspaces are created as independent copies of the YATCC repository to prevent cross-run interference. CMake configuration is performed once per workspace, and build artifacts persist across tasks within a single run.

A.3 Scoring Methodology

Each task’s scoring script (`score.py` or `score-classic.py`) performs:

1. Build the target: `cmake --build build -t taskk`
2. Run the scorer: `cmake --build build -t taskk-score`
3. Parse the output: `score.json` or `score.txt`
4. Extract the total score and per-test details

Scores are normalized to 0–100, with the passing threshold set at 60%.

B Additional Experimental Results

B.1 Full Leaderboard with Process Metrics

Table ?? presents the complete EVOBENCH leaderboard with process metrics for all 18 configurations.

546 **B.2 Ablation: Independent-Task Baseline**

547 To establish a comparison baseline for the ranking shift analysis, we evaluate each model on
548 each task *in isolation* with golden prerequisites provided. This removes the serial dependency
549 effect and measures pure task-level capability.

550 **B.3 Ablation: Weight Sensitivity**

551 We test whether the ranking conclusions depend on the specific weight choice $w =$
552 $[0.05, 0.20, 0.20, 0.15, 0.30, 0.10]$ by evaluating under alternative weight schemes:

- 553 • Uniform weights: $w = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6]$
- 554 • Late-heavy weights: $w = [0.05, 0.05, 0.10, 0.20, 0.30, 0.30]$
- 555 • Early-heavy weights: $w = [0.10, 0.30, 0.30, 0.15, 0.10, 0.05]$

556 **B.4 Ablation: Without Task Guides**

557 We evaluate a subset of models without injecting task-specific documentation into the
558 agent’s context, testing whether the provided guides significantly affect performance.